

Report of I2P II Final Project.

- To start off, the main project is actually built on the Negamax algorithm. The Negamax algorithm is basically Minimax, but our opportunities are the opponent's losses, so the points in which we can get will be the points the opponent lost. In the code, there will be something like "score = same ? raw_score : -raw_score;" which at every recursion, it will maximize or minimize the points that our AI can get or the opponent can get/
- Next up, based on the Negamax algorithm, I implemented Alpha - Beta Pruning. But not the same as the theories, where Alpha least amount of points that we can get, Beta is the largest amount of points that the opponents can give us, and the algorithm will trim off the trees where the leaves surpass the criteria in which $\alpha \leq N \leq \beta$. I implemented the Nega AlphaBeta instead. The logic is basically the same, but instead of just implementing one side, the Nega AlphaBeta will be based on the turns change to help prune the tree. So our top amount, which is Beta will be the least amount that the opponent will try to get (which the amount will be -Beta) and our least amount will be their greatest amount (we are guaranteed to get these amount so they cannot surpass -Alpha) so every recursion, we will swap the values of Alpha and Beta for each participant, after that we will update the alpha value if the current value is higher than the original alpha, and if alpha is $\geq \beta$, that branch will be cut off. (We can understand this by imagining that if a move gives so many points to us, the opponents won't let it happen, and then the moves later in that branch won't be evaluated further)
- After that, we have PVS (Principal Variation Search). Normally, the PVS will assume that the leftmost branch (which, presumably, is the first move) is the best move that can happen. It will AlphaBeta through that node and find the exact value of it, next step it will go through other moves and have Null Windows which force the beta values to only 1 point higher than alpha, and it will test all the moves. If that move's score is ≤ 50 , it will be cut off; otherwise, the PVS will go to that move and run AlphaBeta from there to find the exact score and see if that move is really good. And the Nega PVS version will also have the same exact idea as the original version, but instead of only going from $[\alpha, \alpha+1]$ to $[\alpha, \alpha+1]$, it will have the same recursion logic as the Nega AlphaBeta, which swaps alpha and beta based on the players. And in the code will be something like $[-\alpha - 1, \alpha]$ and looping recursively.
- After implementing PVS, my AI is still losing against Minimax-weak, Minimax-strong and the visible boss, so I have also implemented Quiescence search. Quiescence search is based on Nega AlphaBeta algorithm. After PVS goes to the deepest depth, it will go through the horizon effect at such a time when the depth is ≤ 0 , the AI cannot know if there are any of our pieces are gonna be caught, so the points at such depth will still be high and after certain moves, it will turn out to be that move was so bad. So the Quiescence search is implemented so at every depth ≤ 0 , it will get activated. First, it will get a permission to not capture the opponent's piece, it will take the score of the present, comparing with the upper bound Beta, if it surpasses the Beta, that branch will be cut off immediately. Then, if the present score isn't surpassing the upper bound it will go deeper to that move, the main part is it will only find the capture moves and return the points after all the captures is evaluated until no more capture moves can be found, it will return the score of that moment.

- After implementing Quiescence into NegeMax PVS, I found out that the PVS only focuses on how fast the pruning part, so another problem happened. Since the PVS will assume that the first move is the best move, so what would happen if that move is actually really bad? It would take a lot more time to expand and evaluate other moves to find out which might pass the limit of 2000ms per move. So I ask several questions to ChatGPT and some other AI, and at last, I have the Move Ordering Framework implemented inside. The Move Ordering Framework actually helps sort the moves from best to worst, and the sorting procedure includes 5 priorities and the moves before being searched will undergo these 5 conditions.
- The first one is to search up in a Transposition Table. The Transposition Table was created to store the moves that the PVS has searched through and return them into 4 pieces of information: depth, score, best_move, and flag. The transition table will be called when the eval_ctx function is run and at the end of it. At first, it will find if the move that we are going to evaluate had been searched or not, if yes, and the depth of the searched move is $\geq$ the current depth, it will check the flags which include (TT_EXACT, TT_LOWER, and TT_UPPER), TT_EXACT is to check if the score of the searched move is exact inside the Alpha Beta bound, it will take that score for that move without doing any further search, TT_LOWER is to check if the score is higher than the Beta upper bound, which means the opponent won't let us get that move, it will raise the lower bound Alpha to that score and prune that branch. TT_UPPER is when the score is even lower than the lower bound ALPHA, it will know that this move is so bad and lower the beta bound to that score. Even when the depth requirement is not met. The Transposition Table is still useful; it will provide the best move at the lower depth, which can be useful at deeper depth. And at the end of the eval_ctx, it will store all the categories mentioned above inside a Hash map so we can trace it next time.
- The second priority inside the Move Ordering Framework is actually really short, is the logic for Immediate Win moves. The logic for this algorithm is pretty easy, it will check if any moves can capture the King in the next move or not, if there is, it will return a very large point which in the code is P_MAX - ply (the ply is the moves that was played before this move happens so it tell the AI that is the move is faster or not) the P_MAX is very large (as declare inside the code) so when being returned, the lower bound alpha will be raised to a value which very large and the beta pruning will happen and will reduce a lot of recursion loop. Also, the Immediate win not only run inside the eval_ctx, it will also run inside search, quiescence, which at every loop if it finds a move that can win, it will also cut the tree and save time.
- Next is the MVV-LVA (Most Valuable Victim - Least Valuable Aggressor) as how it called, the algorithm is basically check which capture move is better, the equation of the point for each capture is $10 * \text{capture_piece} - \text{piece_to_capture}$ (is like if we prioritize capturing a high value piece with our low value piece) the algorithm make the pruning faster based on the the equation, because when we "can" capture a high value piece (like a queen) with our low value piece (like a pawn) the score of that move is so high which triggered the beta pruning. Also, it helps the PVS to sort the moves which like if we can capture their rook with our pawn or their pawn with our bishop, it will prioritize capturing rook with pawn.
-